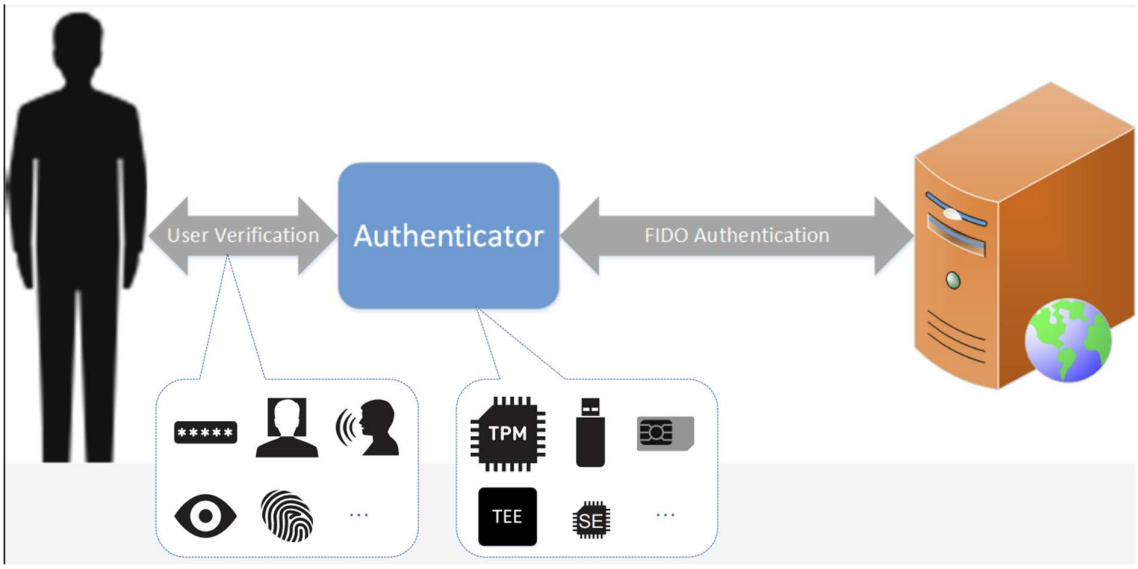
Fido Provider – Developer Guide

The fido provider is a simplified adaption of a Fido U2F client library functionality, as an alternative to the Optional Fido.dll binary library supplied within this GitHub respsitory.



Develop Fido Prrovider

To implement a software-based **FIDO U2F (CTAP1) client provider** in C#, you need to emulate how a physical security key behaves. But there is no standardised solution, the developer is free to choose any implementation as long as it achieves the same end result.

The core responsibility of a U2F client/authenticator is to respond to a server's authentication request by signing a challenge using **EdDsa (with the ed25519 /Edward's curve)**. The default **ed25519 based** application was developed and tested using the NuGet Chaos.NaCl library.

Supporting Routines

The following two functions are fully aligned to the Fido U2F stanard

Cryptographic Workflow

Key Generation

1. The token receives an AppID an infrastructure Secure Identity endpoint.
2. The Token provider generates a secure device key.
3, The token generates a cryptographically secure random number, the nonce
4. The **ed25519 seed** is derived via a KDF rather than being stored as follows:

$$\text{HMAC-SHA256}(\text{AppID} \parallel \text{nonce}, \text{DeviceKey})$$

But the developer can choose any function.

5. The **ed25519** key pair is generated from this seed.
6. The Device Secure Identity is constructed as follows:

```
public string Identity()
{
    // Build public key bytes[]
    return "0199" + Convert.ToHexString(RIPEMD160.Create().ComputeHash(publicKey)).ToLower();
}
```

7. The Public Jason Web (Jwk) key plus Device Identity is sent to the server to be stored against the Secure Identity's account. The Private Key is wiped from the token's memory and never stored.

Authentication or Signature Phase

1. The Token Provider reconstructs the seed from the protected device key and the above KDF.
2. Each transaction between the Device and the Host has a unique signed hash.
The signature is a DER encoded, where **ed25519** r and s are unsigned, big endian numbers, but in little Indian order within this implementation.
3. The Host verifies the DER signature against the stored Jason Web (Jwk) key, the public key is never made available outside of the Device Host infrastructure

Supporting Funtions

```
3 references
public void Enrol(string type)
{
    // Determine technology type
    string relativeUrl;
    if (GetTechType() == TechType.Classic)
        relativeUrl = "User/Register";
    else
        relativeUrl = "api/Register";

    // Extract Function endpoint from User Token
    Uri _baseUri = new Uri(Audience());
    // Obtain Function JwToken
    HttpClient _httpClient = new HttpClient();
    Uri _uri = new Uri(_baseUri, relativeUrl);
    _httpClient.BaseAddress = _baseUri;
    // Add Auth Jwtoken
    string authToken = GetJwToken();
    _httpClient.DefaultRequestHeaders.Add("x-token", authToken);
    _httpClient.DefaultRequestHeaders.Add("x-jwk", GetJwk());
    _httpClient.DefaultRequestHeaders.Add("x-xjwk", GetXJwk());
    string ssign = Sign(GetDeviceSin());
    _httpClient.DefaultRequestHeaders.Add("x-jws-signature", ssign);
    // Get response
    HttpResponseMessage response = _httpClient.GetAsync(_uri).Result;
    if (response.IsSuccessStatusCode)
    {
        TokenRoleType t = GetRoleType();
        // pass
        string json = response.Content.ReadAsStringAsync().Result;
        byte[] bjson = HostDeviceUnProtect(json);
        RecoveryResponse r = JsonSerializer.Deserialize<RecoveryResponse>(bjson);
        SetCounter((uint)r.counter);
        SetJwToken(r.jwToken);
        SetDomain(r.domain); // Optional Domain
        if (string.IsNullOrEmpty(r.pin) == false)
            SetPin(r.pin);
    }
    else
    {
        // fail
        string error = response.Content.ReadAsStringAsync().Result;
        throw new Exception(error);
    }
}
```

```
/// <summary>
/// Sign and return Signature (bytes[]),
/// </summary>
/// <param name="keyHandle"></param>
/// <param name="hashData"></param>
/// <returns></returns>
/// <exception cref="Exception"></exception>
22 references | 0/15 passing
public string Sign(string data)
{
    // Ed25519
    byte[] signature = _edDsa.SignData(data);
    return Base64Url.Encode(signature);
}
```

Within the Fido stanard references to URL are replaced by Secure Identity end points. The only requirement is that these be unique with the scope of the infrastructure.

Supporting Data

Optional: One Link Fido<->Host data structure, cipher text is plaintext, Cipher Encoding set to TLS.

```
public class LinkInfo
{
    9 references
    public TokenRoleType Role { get; set; }
    15 references
    public string SecureIdentity { get; set; }
    23 references
    public string Jwk { get; set; }
    32 references
    public byte[] CipherText { get; set; }
    1 reference
    public byte[] PlainText { get; set; }
    9 references
    public string JwToken { get; set; }
    7 references
    public byte[] Tag { get; set; }
    3 references
    public byte[] WKey { get; set; }
    17 references
    public ulong  CHash { get; set; }
    3 references
    public ulong DHash { get; set; }
    24 references
    public uint Ld { get; set; }  // Last digits
    3 references
    public string Uri { get; set; }
    41 references
    public CipherEncoding Ce { get; set; } // Encoded Support
    34 references
    public Guid Jti { get; set; }
    7 references
    public LinkInfo()
    {

        Ce = CipherEncoding.None; // remove all processing
    }
```

One Link multi-protocol support.
Set to default of TLS or None (requires EcDH X25519 code support).

```
[Flags]
85 references
public enum CipherEncoding
{
    15 references
    None = 0,                   // Only X25519 EcDh no counter, domain or jti hence (deterministic), normally implcit TLS as https Azure hosts
    19 references
    Counter = 1 << 0,       // 1  // X25519 EcDh plus counter
    12 references
    Domain = 1 << 1,        // 2  // Only Domain + Jti random (aad), no counter
    18 references
    Lwe = 1 << 2,           // 4  // Jti random (aad), combined with learning with errors OWF... best for device boot as no prepositioned infastructure required.
    2 references
    Tls = 1 << 3,           // 8  // No One Link support, just Network TLS secruty as Host is always https:
    0 references
    MlKem = 1 << 4,         // 12 // TBD Net10 MLKEM
    5 references
    CounterDomain = Counter | Domain, // 3 // No X25519 only combined Counter and Domain (common cryptographic universals). Best for Military Messaging over One Link as no Public Keys involved only One Cipher
}
```

Device Initial Registration

1. Obtain device Authorisation token from operator.
2. Execute Registration request, using the Local Fido Datastore and the supporting enrol code above.

```
/// <summary>
/// Register Device
/// </summary>
/// <param name="sin">Secure Identity</param>
/// <returns>KeyHandle</returns>
/// <exception cref="Exception"></exception>
1 reference
private void Register(ref DeviceStore ds)
{

    // FIDO AppId->SecureIdentity
    byte[] Appid = Encoding.UTF8.GetBytes(ds.SecureIdentity);
    // Generate registration nounce
    byte[] nounce = CsPrng.SecureRandom(32).ToArray();
    // Derive private key .
    byte[] _keyBytes = DeriveKey(Appid, nounce);
    byte[] KeyHandle = DeriveKeyHandle(nounce, Appid, _keyBytes);
    // Ed25519 signatures
    _edDsa = new EdDsa(_keyBytes);
    // Update DeviceStore
    ds.KeyHandle = KeyHandle;
    ds.JwKey = ExportEd25519ToJwk(_edDsa.PublicKey);
    ds.DeviceSin = DeviceIdentity(_edDsa.PublicKey);

}
```

3. Open the device to process API calls over TLS.